

STUDENT NAME Nurailym Nurlankyzy	STUDENT ID 230103071	HOST CPU / SOC MODEL 13th Gen Intel Core i7-13700HX	OPERATING SYSTEM & KERNEL Windows 11 Version 25H2 OS Build 26200.....
-------------------------------------	-------------------------	--	--

Generative AI & Tooling Policy: You are explicitly authorized to use LLMs (ChatGPT, Claude, GitHub Copilot) to generate code scripts in any language (C, C++, Python, Go, Rust, Java). However: All execution timings, memory counters, and hardware cache metrics must be physically benchmarked on your host machine. Fabricated numbers receive zero credit.

15 Minutes | 20 Points Query your operating system and CPU registers to document

physical core topology, cache boundaries, and vector math extensions.

Terminal Audit Commands: Linux: lscpu && lscpu --caches macOS: sysctl -a grep machdep.cpu Windows: Run wmic cpu get /format:list and inspect Task Manager > Performance > CPU.
--

Hardware Property	Local Measured Value	Hardware Property	
CPU Model & Architecture	13th Gen Intel Core i7-13700HX, x86-64]	Base & Max Boost Clock	2.10 GHz / 5.00 GHz
Physical Hardware Cores	16	Logical Cores (SMT/Threads)	24
L1 Data / Instruction Cache	L1 total: 1.4 MB	L2 Cache (Per Core / Total)	14MB total
L3 Shared Last-Level Cache	30MB	SIMD Vector Extensions	Intel AVX2

Q1.1: Map your host machine's execution modes into Flynn's Taxonomy:

My Intel Core i7-13700HX supports different execution modes depending on the workload. SISD (Single Instruction, Single Data) occurs during ordinary sequential single-thread execution, where one instruction stream operates on one data stream. SIMD (Single Instruction, Multiple Data) is used through vector extensions such as Intel AVX2, allowing one instruction to operate on multiple data elements simultaneously. MIMD (Multiple Instruction, Multiple Data) occurs when multiple CPU cores/threads execute different instruction streams on different data simultaneously. My processor has 16 physical cores and 24 logical processors, allowing it to support multi-threaded MIMD execution. Therefore, the same machine can demonstrate SISD, SIMD, and MIMD depending on the workload.
--

35 Minutes | 30 Points

Write or prompt an AI to generate a compute-bound CPU benchmark parameterized by thread count N. Run 3 iterations per thread count and log wall-clock execution time.

Workload Algorithm: [e.g., Prime Sieve to 5,000,000 / Matrix Multiply]				Language & Runtime: [e.g., C OpenMP / Python multiprocessing / Go]		
Threads (N)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Avg Time TN (s)	Speedup SN = T1/TN	
N = 1 (Baseline)	2.0729 s	2.3422 s	2.1909 S	2.2020 s	1.00×	100.0%
N = 2	1.6365	1.7958	1.7095	1.7139 s	1.285×	64.2%

N = 4	1.1092	1.1876	1.1657	1.1542	1.908×	47.7%
N = 8	0.9406	0.9405	0.8788	0.9200	2.394×	29.9%
N = 16	0.9109	0.9170	0.8629	0.8969	2.455×	15.3%
N = 32	1.1164	1.0638	1.0897	1.0900	2.020×	6.3%

Q2.1: Scaling Saturation & Hardware Contention Analysis:

The benchmark shows that performance improves as the thread count increases from 1 to 16 threads. The best measured performance occurs at **16 threads**, with an average execution time of **0.8969 seconds** and a speedup of approximately **2.455×**. When the thread count increases to 32, execution time increases to **1.0900 seconds**, and speedup decreases to **2.020×**. Therefore, scaling reverses at **32 threads**.

My CPU has **16 physical cores and 24 logical processors**, so using more threads than the available physical cores increases competition for CPU execution resources. At 32 threads, the workload requires more concurrent software threads than the processor's 24 logical processors. This causes additional OS scheduling overhead and contention for shared CPU resources and cache. Consequently, adding more threads does not improve performance and instead makes the benchmark slower.

The results demonstrate that parallel speedup is not unlimited. The workload benefits from additional threads up to approximately 16 threads, after which hardware resource contention and parallelization overhead dominate.

TASK 3: The Unsynchronized Shared Counter & Race Condition Trap 25 Minutes | 25 Points

Spawn 10 parallel threads. Each thread executes a loop incrementing a shared global integer 1,000,000 times without atomic operations or locks. Theoretical expected total: $10 \times 1,000,000 = 10,000,000$. Execute 10 consecutive runs.

Run	Measured Output	Error (10^7 - Act)	Run	Measured Output	
#1	1,000,838	8,999,162	#6	1,000,213	8,999,787
#2	1,000,214	8,999,786	#7	1,000,154	8,999,846
#3	1,000,243	8,999,757	#8	1,002,569	8,997,431
#4	1,000,199	8,999,801	#9	1000302	8,999,698
#5	1,000,288	8,999,712	#10	1000112	8,999,888

Q3.1: Read-Modify-Write (RMW) Window

- Core 1: Load **100**
- Core 2: Load **100**
- Core 1: Add **1** → **101**
- Core 2: Add **1** → **101**
- Core 1: Store **101**
- Core 2: Store **101**

Although two increments were performed, the final value is only **101** instead of **102**. The update from Core 1 is overwritten by Core 2. This is called a **lost update**.

Because the Load, Add, and Store instructions are not atomic as a group, another

Q3.2: The Synchronization Penalty

Unlocked = _____ ms vs. Locked = _____ ms

The race-free synchronized version is substantially slower because every thread must acquire and release a mutex/lock before modifying the shared counter. Threads may also have to wait when another thread owns the lock. This introduces synchronization overhead and reduces parallelism because only one thread can enter the critical section at a time.

Therefore, the lock prevents lost updates and guarantees the correct result, but the additional lock acquisition, release, waiting, and thread contention increase

core can access the shared memory between these operations. Repeating this interleaving millions of times causes severe value corruption and produces a final result much smaller than the theoretical 10,000,000.	execution time.
--	-----------------

TASK 4: Empirical Amdahl Verification vs. Gustafson Scaling Horizon 15 Minutes | 15 Points

Use your empirical single-thread time (T1) and two-thread time (T2) from Task 2 to derive the parallel fraction p of your workload, then evaluate theoretical performance bounds.

1. Parallel Fraction Derivation (p) Formula: $p = 2 \times (T1 - T2) / T1$ T1 = 2.2020 s T2 = 1.7139 s Calculated p = 0.4433 (decimal between 0.0 and 1.0)	2. Amdahl Theoretical Speedup Ceiling Formula: $S_{max} = 1 / (1 - p)$ Sequential fraction (1 - p) = 0.5567 Max Theoretical Speedup ($N \rightarrow \infty$) = 1.796x
3. Amdahl 64-Core Projection Formula: $S_{64} = 1 / [(1 - p) + (p / 64)]$ Projected Speedup on 64 Cores = 1.774x	4. Gustafson Scaled Speedup (Weak Scaling) Formula: $S_{Gustafson} = (1 - p) + (p \times 64)$ Projected Scaled Speedup (64x Problem) = 28.928x

Q4.1: Strong vs. Weak Scaling Analysis:

Amdahl's model assumes a fixed problem size. Therefore, the sequential portion remains a bottleneck even when additional processor cores are added. In this case, the sequential fraction is approximately 0.5567, which places a strong upper limit on the speedup. Even with 64 cores, Amdahl's model predicts only approximately 1.774x speedup. Gustafson's model assumes weak scaling, where the problem size grows as more processors are added. As the workload becomes larger, the parallel portion can dominate the total execution time, allowing additional processors to perform substantially more useful work. Therefore, Gustafson's model predicts approximately 28.928x speedup for a 64x scaled problem. Thus, Amdahl's Law emphasizes the bottleneck created by the sequential portion of a fixed-size problem, while Gustafson's Law emphasizes the ability to solve proportionally larger problems using additional processors.

Submission Guidelines & Deliverables Checklist:

- Complete all 4 tables and technical reflection prompts directly on this document (or submit as an annotated Word / PDF).
- Attach a single terminal screenshot verifying Task 2 executing across active threads (e.g., htop, top, or Task Manager CPU pane).
- Upload the final .docx / PDF to the course portal before the conclusion of the 90-minute lab period.